

Protokoll

Allgemeine Programmübersicht

Das Ziel des Programms ist die Verwaltung eines gewichteten Graphen, der ein Verkehrsnetz (z.B. Wiener Linien) darstellt. Der Fokus liegt auf dem Einlesen von Daten aus einer Textdatei, dem Aufbau der Graphstruktur sowie der Suche eines Pfades zwischen zwei Stationen.

Dabei wird ein einfacher Suchalgorithmus (rekursive Tiefensuche) verwendet, um einen möglichen Weg vom Startknoten zum Zielknoten zu finden und anschließend auszugeben.

Datenstruktur

Die zentrale Datenstruktur ist ein gewichteter Graph.

Jede Station wird als Knoten (`Station`) gespeichert.

Jeder Knoten besteht aus:

- einem Stationsnamen (`name`)
- einer Liste von Nachbarn (`neighbors`)

Jeder Nachbar wird als `Edge` gespeichert:

- Zielstation (`to`)
- Gewicht der Kante (`weight`)
- Liniename (`line`)

Der gesamte Graph wird in einem Vektor gespeichert:

- `std::vector<Station> graph`

Die Verbindungen zwischen den Stationen werden als **Adjazenzliste** dargestellt.

Vorteile dieser Struktur:

- einfache Erweiterung beim Einlesen
- effizientes Speichern vieler Verbindungen
- schnelles Durchlaufen der Nachbarn bei der Suche
-

Einlese- und Aufbauprozess

Die Funktion `readFromFile()` liest den Graphen aus einer Textdatei ein.

Die Datei besteht aus Zeilen, die folgendes Format haben:

- Liniename (z.B. U1, U2, 1, 5)
- danach abwechselnd Stationsnamen (in Anführungszeichen)
- und Kostenwerte zwischen den Stationen

Beim Einlesen wird:

- die Zeile zuerst beim Doppelpunkt getrennt
- der Liniename gespeichert
- anschließend werden Stationen und Kosten aus der Zeile extrahiert

Die Stationen werden dabei:

- aus dem Text zwischen " " gelesen
- als String gespeichert
- und in den Graph eingefügt, falls sie noch nicht existieren

Danach werden Kanten erzeugt:

- jede Station wird mit der nächsten verbunden
- zusätzlich wird die Gegenrichtung eingefügt (ungerichteter Graph)

Suchalgorithmus (Pfadsuche)

Für die Pfadsuche wird eine rekursive Tiefensuche (DFS) verwendet.

Die Funktion `find_simple_path()` arbeitet wie folgt:

- Startknoten wird markiert als besucht
- alle Nachbarn werden rekursiv überprüft
- bereits besuchte Knoten werden nicht erneut besucht
- wenn der Zielknoten erreicht wird, wird die Suche beendet

Während der Suche wird zusätzlich gespeichert:

- welcher Knoten von welchem vorherigen Knoten erreicht wurde (`parent`)

Damit kann später der Pfad rekonstruiert werden.

Pfadausgabe

Die Funktion `printPath()` rekonstruiert den gefundenen Weg.

Dabei wird:

- vom Zielknoten rückwärts über `parent` zum Startknoten gegangen
- der Pfad in einem Vektor gespeichert
- anschließend in richtiger Reihenfolge ausgegeben

Das Ergebnis ist eine klare Darstellung des Weges von Start bis Ziel.

Aufwandsabschätzung

Einlesen der Datei (**readFromFile**)

Jede Zeile wird einmal gelesen und verarbeitet.

- Best-case: $O(n)$
- Average-case: $O(n)$
- Worst-case: $O(n)$

(n = Anzahl der Kanten in der Datei)

Stationssuche (**findStation**)

Die Suche erfolgt linear über alle Knoten.

- Best-case: $O(1)$ (erste Station gefunden)
- Average-case: $O(n)$
- Worst-case: $O(n)$
-

Einfügen einer Kante (**addEdge**)

Für jede Verbindung wird zuerst die Station gesucht und dann eine Kante eingefügt.

- Best-case: $O(1)$
- Average-case: $O(n)$
- Worst-case: $O(n)$

Da dies für alle Kanten passiert, ergibt sich insgesamt eine lineare bis quadratische Abhängigkeit je nach Graphgröße.

Tiefensuche (**find_simple_path**)

Die Suche besucht jeden Knoten höchstens einmal.

- Best-case: $O(1)$ (Ziel direkt erreichbar)
- Average-case: $O(n + m)$
- Worst-case: $O(n + m)$

(n = Knoten, m = Kanten)

Pfadrekonstruktion (**printPath**)

Der Pfad wird vom Ziel zum Start rückwärts aufgebaut.

- Best-case: $O(1)$
- Average-case: $O(k)$

- Worst-case: $O(k)$

(k = Länge des gefundenen Pfades)

Gesamte Pfadsuche

Die Gesamtlaufzeit ergibt sich aus:

1. Einlesen des Graphen $\rightarrow O(n + m)$
2. Tiefensuche $\rightarrow O(n + m)$
3. Pfadausgabe $\rightarrow O(k)$

Gesamt:

- Best-case: $O(n)$
- Average-case: $O(n + m)$
- Worst-case: $O(n + m)$